

Многопоточность в Python. Библиотеки `threading` и `multiprocessing`.

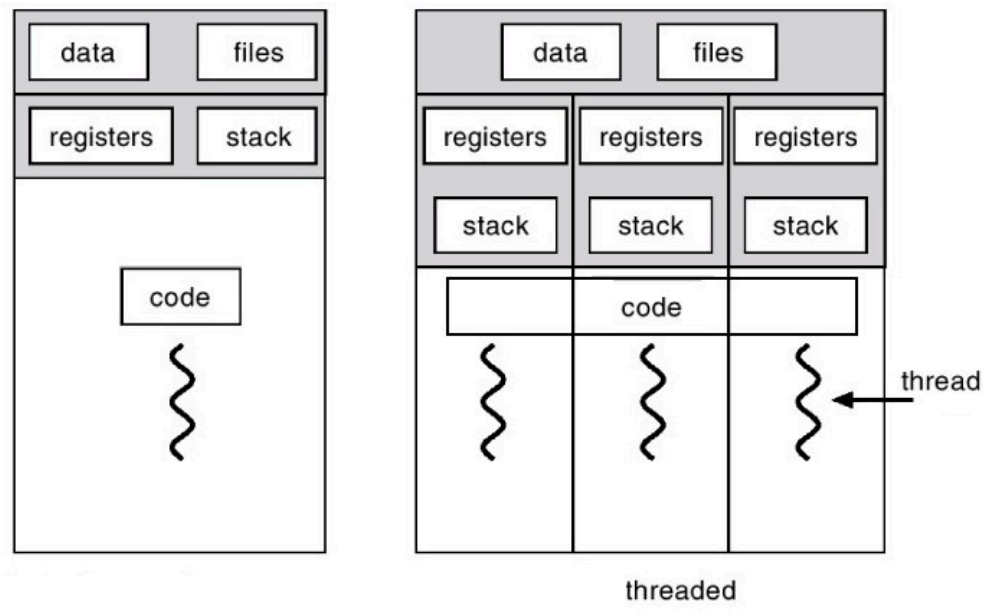
Процесс и поток

Процесс - исполняемый экземпляр какой-либо программы. Каждый процесс состоит из следующих элементов:

- образ машинного кода;
- область памяти, в которую включается исполняемый код, данные процесса (входные и выходные данные), стек вызовов и куча (для хранения динамически создаваемых данных);
- дескрипторы операционной системы (например, файловые дескрипторы);
- состояние процесса.

В целях стабильности и безопасности, в современных операционных системы каждый процесс имеет прямой доступ только к своим собственным ресурсам. Доступ к ресурсам другого процесса возможен через межпроцессное взаимодействие (например, посредством файлов, при помощи именованных и неименованных каналов и другие).

Сам процесс может быть разделен на так называемые потоки. Поток (поток выполнения, `thread`) - наименьшая единица обработки, исполнение которой может быть назначено ядром операционной системы. В отличие от нескольких процессов, потоки существуют внутри одного процесса и имеют доступ к ресурсам этого процесса. Каждый поток обладает собственным набором регистров и собственным стеком вызова, но доступ к ним имеют и другие потоки.



При работе с потоками стоит учесть несколько моментов:

- одно ядро процессора в один момент может исполнять только один поток;
- потоки одного процесса могут исполняться физически одновременно (на разных ядрах);
- бессмысленно порождать потоков больше, чем у вас есть ядер.

Потоки имеют несколько применений. Первое - ускорение работы программы. Ускорение достигается за счет параллельного выполнения независимых друг от друга вычислений. Например, при численном интегрировании область интегрирования может быть разбита на 3 участка. На каждый участок создается свой поток, в котором численно вычислится интеграл для конкретного участка. Второе - независимое исполнение операций. Отличие этого случая от первого хорошо видно на следующем примере. Пусть есть приложение с графическим интерфейсом, где весь код выполняется в одном потоке. При выполнении какой-нибудь долгой операции (например, копирование файла) интерфейс приложения просто перестанет отвечать до тех пор, пока долгий процесс не завершится. В таком случае в один поток помещается работа графического интерфейса, в другой - остальные вычисления. В таком случае интерфейс позволит проводить другие операции даже во время выполнения долгой операции в другом потоке (например, заполнение прогресс бара в процессе копирования файла).

threading

В Python работа с потоками осуществляется при помощи стандартной библиотеки `threading`. В библиотеке представлен класс `Thread` для создания потока выполнения. Задание исполняемого кода в отдельном потоке возможно двумя способами:

- передача исполняемого объекта (функции) в конструктор класса;
- переопределение функции run() в классе-наследнике.

После того, как объект создан, поток запускается путем вызова метода start(). Рассмотрим простой пример:

```
import threading
import sys

def thread_job(number):
    print('Hello {}'.format(number))
    sys.stdout.flush()

def run_threads(count):
    threads = [
        threading.Thread(target=thread_job, args=(i,))
        for i in range(0, count)
    ]
    for thread in threads:
        thread.start() # каждый поток должен быть запущен
    for thread in threads:
        thread.join() # ожидаем исполнения всех потоков

run_threads(4)
print('finish')
```

Конструктор класса Thread имеет следующие аргументы:

- group должно быть None; зарезервировано для будущих реализаций Python 3;
- target является исполняемым объектом (по умолчанию равен None, ничего не исполняется);
- name обозначает имя потока (по умолчанию имя генерируется автоматически);
- args - кортеж аргументов для исполняемого объекта;
- kwargs - словарь именованных аргументов для исполняемого объекта;
- daemon равное True обозначает служебный поток (служебные потоки завершаются принудительно при завершении процесса); по умолчанию False.

В Python выполнение программы заканчивается, когда все неслужебные потоки завершены. Модифицировав программу выше, мы все еще получим корректно работающий код:

```
import threading
import sys
import time

def thread_job(number):
    time.sleep(2) # "усыпляем" поток на 2 сек
    print('Hello {}'.format(number))
    sys.stdout.flush()

def run_threads(count):
    threads = [
        threading.Thread(target=thread_job, args=(i,))
        for i in range(1, count)
    ]
    for thread in threads:
        thread.start() # каждый поток должен быть запущен

run_threads(1)
print(finish)
```

Как можно увидеть, программа завершается без ошибок (с кодом 0), но теперь строка "finish" печатается раньше строки "Hello 0", т.к. главный поток теперь не ждет завершения работы других потоков. Метод `join()` используется для блокирования исполнения родительского потока до тех пор, пока созданный поток не завершится. Это нужно в случаях, когда для работы потока-родителя необходим результат работы потока-потомка. Вспомним пример с численным интегрированием. Вычисление итогового значения интеграла выполняется в главном потоке, но это возможно только после завершения вычислений в побочных потоках. В таком случае главный поток нужно просто приостановить до тех пор, пока не завершатся все побочные потоки. Метод `join()` может принимать один аргумент - таймаут в секундах. Если таймаут задан, `join()` блокирует работу на указанное время. Если по истечении времени ожидаемый поток не будет завершен, `join()` все равно разблокирует работу потока, вызвавшего его. Проверить, выполняется ли поток можно методом `is_alive()`. Подробнее ознакомиться с функционалом библиотеки можно в официальной документации по [threading](#).

Упражнение №1

Запустите следующий код. В чем проблема данного кода? Всегда ли `counter = 10` после исполнения кода программы?

```
import threading
import sys

def thread_job():
    global counter
    old_counter = counter
    counter = old_counter + 1
    print('{} '.format(counter), end='')
    sys.stdout.flush()

counter = 0
threads = [threading.Thread(target=thread_job) for _ in range(10)]
for thread in threads:
    thread.start()
for thread in threads:
    thread.join()
print(counter)
```

Демонстрация "проблемности" кода:

```
import threading
import random
import time
import sys

def thread_job():
    global counter
    old_counter = counter
    time.sleep(random.randint(0, 1))
    counter = old_counter + 1
    print('{} '.format(counter), end='')
    sys.stdout.flush()

counter = 0
threads = [threading.Thread(target=thread_job) for _ in range(10)]
for thread in threads:
    thread.start()
for thread in threads:
```

```
thread.join()
print(counter)
```

Почему так происходит? Есть несколько возможных решений этой проблемы.

```
import threading
import random
import time
import sys

def thread_job():
    lock.acquire() # mutex
    global counter
    old_counter = counter
    time.sleep(random.randint(0, 1))
    counter = old_counter + 1
    print('{} '.format(counter), end='')
    sys.stdout.flush()
    lock.release()

lock = threading.Lock()
counter = 0
threads = [threading.Thread(target=thread_job) for _ in range(10)]
for thread in threads:
    thread.start()
for thread in threads:
    thread.join()
print(counter)
```

```
import threading
import random
import time
import sys

def thread_job():
    with lock:
        global counter
        old_counter = counter
        time.sleep(random.randint(0, 1))
        counter = old_counter + 1
        print('{} '.format(counter), end='')
```

```

sys.stdout.flush()

lock = threading.Lock()
counter = 0
threads = [threading.Thread(target=thread_job) for _ in range(10)]
for thread in threads:
    thread.start()
for thread in threads:
    thread.join()
print(counter)

```

Вариант с контекстным менеджером более предпочтителен. Вспомните работу с файлами при помощи with. По завершении with файл автоматически закрывался. В данном случае похожая ситуация. Для того, чтобы запретить нескольким потокам параллельно выполнять некоторые участки кода, мы используем Lock (в UNIX системах более известен как мьютекс (mutex)). Мьютекс может быть в двух состояниях: свободен и заблокирован. Если какой-либо поток пытается заблокировать уже заблокированный мьютекс, то поток блокируется до тех пор, пока мьютекс не освободится. Причем если несколько потоков претендует на блокирование мьютекса, то потоки просто выстраиваются в очередь. Главная проблема - не освобожденный мьютекс. Отсутствие строчки lock.release() может повесить остальные потоки в бесконечное ожидание. Контекстный менеджер позволит избежать этой проблемы. Как только он закончится, все захваченные им ресурсы будут освобождены, в том числе мьютекс.

Упражнение №2

Иногда бывает нужно узнать доступность набора ip адресов. Неэффективный вариант представлен ниже. Реализуйте то же самое, но используя threading.

```

import os, re

received_packages = re.compile(r"(\d) received")
status = ("no response", "alive but losses", "alive")

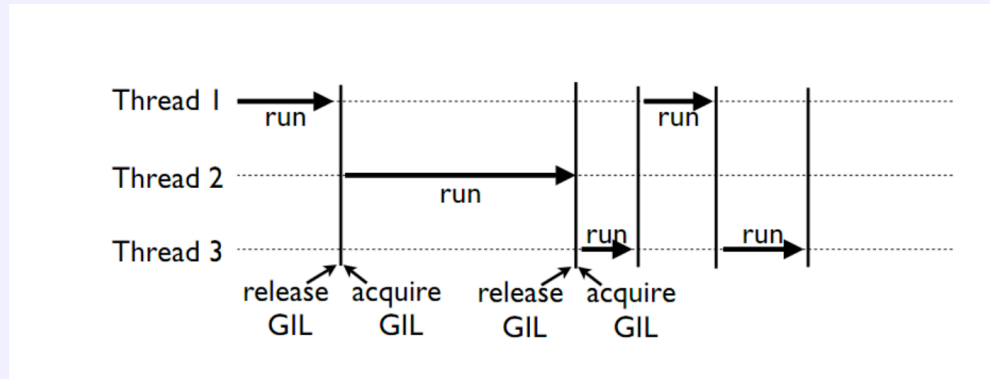
for suffix in range(20, 30):
    ip = "192.168.178." + str(suffix)
    ping_out = os.popen("ping -q -c2 " + ip, "r") # получение вердикта
    print("... pinging ", ip)
    while True:
        line = ping_out.readline()
        if not line:
            break
        n_received = received_packages.findall(line)

```

```
if n_received:
    print(ip + ": " + status[int(n_received[0])])
```

Global Interpreter Lock (GIL)

CPython - популярная реализация интерпретатора - имеет встроенный механизм, который обеспечивает выполнение ровно одного потока в любой момент времени. GIL облегчает реализацию интерпретатора, защищая объекты от одновременного доступа из нескольких потоков. По этой причине, создание нескольких потоков не приведет к их одновременному исполнению на разных ядрах процессора.



Однако, некоторые модули, как стандартные, так и сторонние, созданы для освобождения GIL при выполнении тяжелых вычислительных операций (например, сжатие или хеширование). К тому же, GIL всегда свободен при выполнении операций ввода-вывода.

Упражнение №3

Написать программу, которая будет находить сумму чисел массива с использованием N потоков. Запустить с разным параметром N. Убедиться, что несмотря на увеличение N, ускорения подсчета не происходит. Причина этому - GIL. В Python **вычисления** распараллеливать бессмысленно. Замерить время работы можно с помощью библиотеки time (ответ в секундах):

```
start = time.time()
# код, время работы которого надо замерить
print(time.time() - start)
```

Упражнение №4

Запустите на исполнение, замерив время работы. Перепишите с помощью потоков и опять замерьте время.

При отсутствии доступа к интернету укажите доступные адреса urls ниже. К примеру:

http://cs.mipt.ru/advanced_python/lessons/lab1.html

http://cs.mipt.ru/advanced_python/lessons/lab2.html и т.д.

```
import urllib.request
import time

urls = [
    'https://www.yandex.ru', 'https://www.google.com',
    'https://habrahabr.ru', 'https://www.python.org',
    'https://isocpp.org',
]

def read_url(url):
    with urllib.request.urlopen(url) as u:
        return u.read()

start = time.time()
for url in urls:
    read_url(url)
print(time.time() - start)
```

Потоки очень уместны, если в коде есть блокирующие операции (ввод-вывод, сетевые взаимодействия). Также, удобно разбивать логические процессы по потокам (анимация, графический интерфейс, и тд).

[multiprocessing](#)

Библиотека multiprocessing позволяет организовать параллелизм вычислений за счет создания подпроцессов. Т.к. каждый процесс выполняется независимо от других, этот метод параллелизма позволяет избежать проблем с GIL. Предоставляемый библиотекой API схож с тем, что есть в threading, хотя есть уникальные вещи. Создание процесса происходит путем создания объекта класса Process. Аргументы конструктора аналогичны тем, что есть в конструкторе Thread. В том числе аргумент daemon позволяет создавать служебные процессы. Служебные процессы завершаются вместе с родительским процессом и не могут порождать свои подпроцессы.

Простой пример работы с библиотекой:

```
from multiprocessing import Process
```

```
def f(name):  
    print('hello', name)  
  
if __name__ == '__main__':  
    p = Process(target=f, args=('bob',))  
    p.start()  
    p.join()
```

Чтобы убедиться, что каждый процесс имеет свой ID, запустите пример:

```
from multiprocessing import Process  
import os  
  
def info(title):  
    print(title)  
    print('module name:', __name__)  
    print('parent process:', os.getppid())  
    print('process id:', os.getpid())  
  
def f(name):  
    info('function f')  
    print('hello', name)  
  
if __name__ == '__main__':  
    info('main line')  
    p = Process(target=f, args=('bob',))  
    p.start()  
    p.join()
```

Старайтесь не забывать про конструкцию `__name__ == '__main__'`. Это надо для того, чтобы ваш модуль можно было безопасно подключать в другие модули и при этом не создавались новые процессы без вашего ведома.

Упражнение №5

Запустите код. Попробуйте объяснить, почему LIST - пуст.

```
import multiprocessing
```

```
def worker():
    LIST.append('item')

LIST = []

if __name__ == "__main__":
    processes = [
        multiprocessing.Process(target=worker)
        for _ in range(5)
    ]
    for p in processes:
        p.start()
    for p in processes:
        p.join()
    print(LIST)
```

Общение между процессами

multiprocessing предоставляет два вида межпроцессного обмена данными: очереди и каналы данных (pipe).

Очереди (класс Queue) аналогичны структуре данных "очередь", рассмотренной вами в курсе алгоритмов.

```
from multiprocessing import Process, Queue

def f(q):
    q.put([42, None, 'hello'])

if __name__ == '__main__':
    q = Queue()
    p = Process(target=f, args=(q,))
    p.start()
    print(q.get())    # выводит "[42, None, 'hello']"
    p.join()
```

Класс Pipe отвечает за канал обмена данными (по умолчанию, двунаправленный), представленный двумя концами, объектами класса Connection. С одним концом канала работает родительский процесс, а с другим концом - подпроцесс.

```
from multiprocessing import Process, Pipe
```

```
def f(conn):
    conn.send([42, None, 'hello'])
    conn.close()

if __name__ == '__main__':
    parent_conn, child_conn = Pipe()
    p = Process(target=f, args=(child_conn,))
    p.start()
    print(parent_conn.recv())  # выводим "[42, None, 'hello']"
    p.join()
```

Еще один вид обмена данными может быть достигнут путем записи/чтения обычных файлов. Чтобы исключить одновременную работу двух процессов с одним файлом, в библиотеке есть классы аналогичные threading.

```
from multiprocessing import Process, Lock

def f(l, i):
    l.acquire()
    try:
        print('hello world', i)
    finally:
        l.release()

if __name__ == '__main__':
    lock = Lock()
    for num in range(10):
        Process(target=f, args=(lock, num)).start()
```

Подробнее ознакомиться с функционалом библиотеки можно в официальной документации по [multiprocessing](#).

Класс Pool в multiprocessing

Класс Pool - удобный механизм распараллеливания выполнения функций, распределения входных данных по процессам и т.д.

Наиболее интересные функции: Pool.apply, Pool.map, Pool.apply_async, Pool.map_async.

apply, map работают аналогично питоновским built-in apply, map.

Как работает Pool можно понять на примере:

```
from multiprocessing import Pool

def cube(x):
    return x**3

if __name__ == "__main__":
    pool = Pool(processes=4) # создаем пул из 4 процессов
    # в apply можно передать несколько аргументов
    results = [pool.apply(cube, args=(x,)) for x in range(1,7)] # раскидываем числа от 1 до 7 по 4 процессам
    print(results)

    pool = Pool(processes=4)
    # то же самое, но с тар. разбивает итерируемый объект (range(1,7)) на chunks и раскидывает аргументы по процессам
    results = pool.map(cube, range(1,7))
    print(results)
```

map, apply - блокирующие вызовы. Главная программа будет заблокирована, пока процесс не выполнит работу.

map_async, apply_async - неблокирующие. При их вызове, они сразу возвращают управление в главную программу (возвращают ApplyResult как результат). Метод get() объекта ApplyResult блокирует основной поток, пока функция не будет выполнена.

```
pool = mp.Pool(processes=4)
results = [pool.apply_async(cube, args=(x,)) for x in range(1,7)]
output = [p.get() for p in results]
print(output)
```

Упражнение №6*

Для этого упражнения скачайте архив [viterbi_mp.zip](#) с кодом и необходимыми данными.

Рассмотрим следующую задачу. Положение мобильного робота на двумерной карте может быть представлено тремя числами: x , y и направлением θ . Точное положение робота нам не известно. В связи с этим мы строим N гипотез о его положении, сумма их вероятностей равна 1. В процессе движения робота некоторые гипотезы исчезали, а некоторые порождали новые. Однако в каждый момент времени количество гипотез - константа. Известно, какая гипотеза из какой была порождена.

Представленный (и слегка упрощенный) выше метод оценки положения робота множеством гипотез называется фильтром частиц, а сами гипотезы называются частицами. Фильтр частиц используется для оценки положения робота в процессе его движения. Вспомним, что в процессе работы некоторые частицы погибают, а некоторые порождают другие. Переходы между частицами образуют граф перехода. Используя этот граф, можно оценить траекторию робота с некоторой точностью.

Задача: необходимо восстановить траекторию движения робота. Есть несколько способов приближенно решить данную задачу. Один из способов - восстановить наиболее вероятную траекторию. Для этого воспользуемся алгоритмом [Витерби](#), одним из алгоритмов динамического программирования.

Пусть у нас было T моментов времени. На каждом моменте времени t мы для каждой частицы, существующей в момент времени t , выбираем наиболее вероятный переход из какой-нибудь частицы с момента времени $t-1$. Тогда ответом будет - argmax по вероятности среди всех частиц в последний момент времени. Однако, сам алгоритм довольно медленный. Его асимптотика $O(T * N^2)$.

В архиве вам предоставлен код в файле `generate_viterbi_trajectory.py`. Однако, он написан без распараллеливания. Ваша задача - распараллелить код, используя `multiprocessing`. Файл `graph.ldj` представляет собой текстовый файл, где каждая строка в формате JSON. Каждая строка представляет собой один момент времени. В этом задании вам предлагаются первые 10 моментов времени движения робота. В каждый момент времени количество частиц $n = 2000$. Файл `localization_config.json` - файл конфигурации, содержащий параметры с которыми происходила генерация графа. Файл `true_trajectory.json` содержит массив троек чисел (x, y, θ) , построенный нераспараллеленным алгоритмом. Вам надо будет сравнить полученную вами траекторию с данной при помощи скрипта `correspond_trajectories.py`. Для тех, кто хочет попробовать свой код на больших данных, используйте файл `full_graph.ldj`, который содержит порядка 1700 строк.

[Архив с файлом](#).

Не забудьте замерить время работы. Примерное время работы на моем компьютере для 10 строк в 1 процесс - 300 сек.